

OBJECT DETECTION

A Beginner-Friendly Learning Guide

Definitions | Hinglish Explanation | Algorithms | Metrics | Python Example | Mini Project

Who is this for?

Beginners learning computer vision, machine learning, Python, YOLO, or interview fundamentals. No advanced mathematics is required.

Prepared for: Leelamaya

Date: 14 July 2026

Contents

1. What Object Detection Means
2. Detection Output: Box, Class, and Confidence
3. How an Object Detector Works
4. Important Model Families and Algorithms
5. Dataset Preparation and Training Workflow
6. IoU and Non-Maximum Suppression
7. Evaluation Metrics: Precision, Recall, AP, and mAP
8. Python Example with Ultralytics YOLO
9. Real-World Use Cases, Challenges, and Best Practices
10. Mini Project and Interview Revision

Learning outcome

By the end, you should be able to explain object detection clearly, read prediction results, understand the main metrics, and run a pretrained detector on an image.

Learning Roadmap

Stage	What to understand	Suggested practice
Foundation	Images, pixels, labels, bounding boxes	Draw boxes manually on 5 images
Inference	Load a pretrained detector and predict	Test on images and videos
Evaluation	IoU, confidence, precision, recall, mAP	Compare correct and wrong detections
Training	Annotations, train/validation split, augmentation	Train on a small custom dataset
Deployment	Latency, model size, hardware, monitoring	Build a simple API or webcam demo

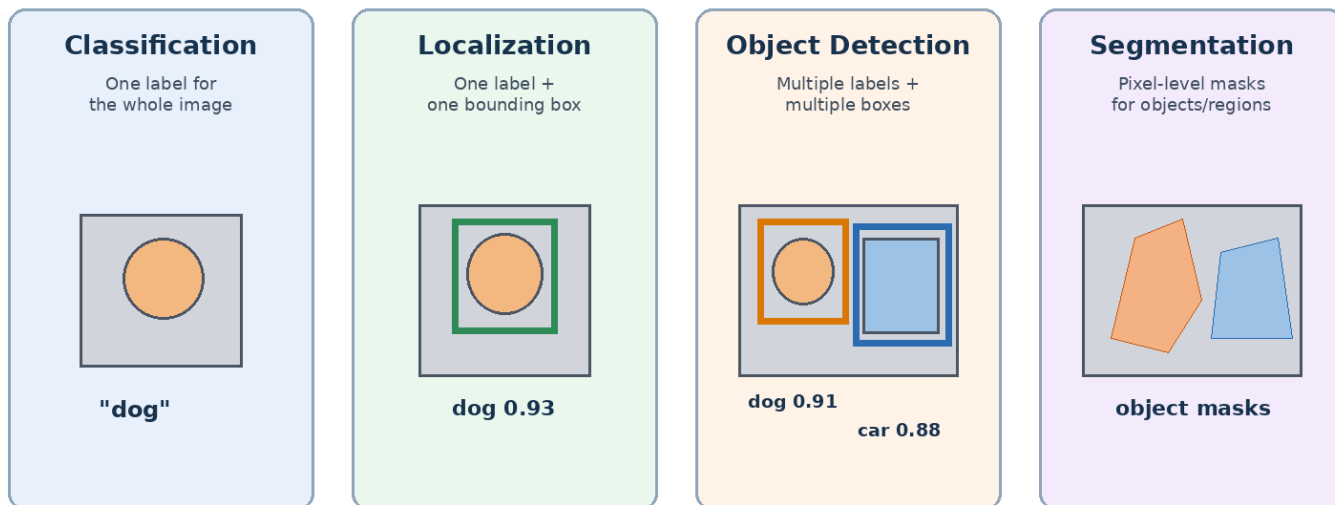
1. What Is Object Detection?

Object detection is a computer vision task that identifies which objects are present in an image or video and locates each object using a bounding box. A detector normally returns a class label, box coordinates, and a confidence score for every detected object.

Hinglish explanation

Object detection ka matlab sirf image me kya hai batana nahi hota. Model ko yeh bhi batana hota hai ki object image ke kis area me hai. Isliye output me class name ke saath bounding box aur confidence score milta hai.

Computer Vision Tasks: What does each task return?



Why detection is harder than classification

- An image may contain zero, one, or many objects.
- Objects can appear at different sizes, angles, lighting conditions, and distances.
- The model must solve two problems together: classification and localization.
- Overlapping objects, occlusion, blur, and complex backgrounds increase difficulty.

Core vocabulary

Term	Meaning
Bounding box	A rectangle that encloses an object, usually represented by corner coordinates or center, width, and height.
Class label	The category predicted for an object, such as person, car, dog, or bottle.
Confidence score	The model's estimated confidence that a predicted box contains a particular class.
Ground truth	The correct human-provided annotation used for training or evaluation.
Inference	Running a trained model on new data to generate predictions.

2. Detection Output: Box, Class, and Confidence

A single detection is commonly represented as:

```
class = "car"  
confidence = 0.92  
box = [x_min, y_min, x_max, y_max]
```

Bounding-box coordinate formats

Format	Values	Common use
XYXY	x_min, y_min, x_max, y_max	Easy for drawing rectangles and calculating IoU.
XYWH	x_min, y_min, width, height	Used by several annotation tools and datasets.
CXCYWH	center_x, center_y, width, height	Common in YOLO-style labels, often normalized between 0 and 1.

Confidence threshold

A confidence threshold controls which predictions are kept. For example, with a threshold of 0.25, predictions below 0.25 are discarded. Increasing the threshold usually reduces false positives but may also remove true objects with lower confidence.

Simple rule

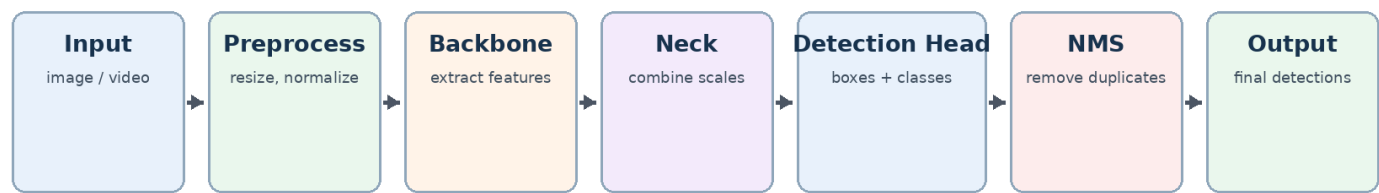
Low threshold = more detections, including more wrong detections. High threshold = fewer detections, but some real objects may be missed.

Example prediction table

Detected class	Confidence	Box coordinates	Interpretation
person	0.96	[84, 42, 302, 518]	Strong person detection.
bicycle	0.81	[245, 210, 510, 488]	Likely correct bicycle detection.
dog	0.19	[520, 300, 610, 430]	Usually removed when confidence threshold is 0.25.

3. How an Object Detector Works

Typical Object Detection Inference Pipeline



Training adds labeled boxes, loss calculation, backpropagation, and repeated weight updates.

Step-by-step explanation

- 11. Input: The model receives an image, video frame, webcam feed, or stream.
- 12. Preprocessing: The input is resized, padded if required, converted into numeric tensors, and normalized.
- 13. Backbone: A convolutional or transformer-based network extracts features such as edges, textures, shapes, and object parts.
- 14. Neck: Features from multiple scales are combined so the model can detect both small and large objects.
- 15. Detection head: The network predicts candidate boxes, class scores, and confidence values.
- 16. Post-processing: Low-confidence predictions and duplicate overlapping boxes are removed, usually using NMS.
- 17. Output: Final boxes and labels are returned or drawn on the image.

Training vs inference

Training	Inference
Uses labeled images and ground-truth boxes.	Uses a trained model on new images.
Calculates losses and updates model weights.	Does not update weights.
Repeated for many epochs.	Usually optimized for speed and memory.
Goal: learn patterns that generalize.	Goal: produce useful predictions.

4. Important Model Families and Algorithms

One-stage vs two-stage detectors

Detector family	How it works	Strengths	Trade-offs
One-stage	Predicts boxes and classes directly in one main pass.	Fast; suitable for real-time applications.	May need careful tuning for tiny or crowded objects.
Two-stage	First proposes possible regions, then classifies and refines them.	Often strong localization and accuracy.	Usually slower and heavier.

Common algorithms

Algorithm	Family	Key idea	Typical suitability
YOLO	One-stage	Treats detection as a direct prediction problem over the image.	Real-time video, practical applications, edge deployment.
SSD	One-stage	Predicts objects at multiple feature-map scales.	Fast detection with a relatively simple architecture.
RetinaNet	One-stage	Uses focal loss to reduce the effect of easy background examples.	Datasets with strong class imbalance.
Faster R-CNN	Two-stage	Uses a region proposal network followed by classification and box refinement.	Accuracy-focused systems where speed is less critical.
DETR	Transformer-based	Uses object queries and set prediction instead of traditional anchor/NMS-heavy design.	Research and transformer-based detection pipelines.

Interview answer

YOLO is popular because it offers a strong speed-accuracy balance and can process images, videos, and live streams. Faster R-CNN is a two-stage detector and is generally chosen when localization quality matters more than real-time speed.

Model-size choice

- Small/nano model: low latency and lower resource usage, but lower accuracy.
- Medium model: balanced speed and accuracy for many applications.
- Large model: potentially better accuracy, but higher GPU memory and latency.
- The best model is decided by measured accuracy, latency, hardware, and business requirements - not only benchmark score.

5. Dataset Preparation and Training Workflow

What a training dataset contains

- Images representing real operating conditions.
- A class list, such as helmet, person, vehicle, or product.
- Bounding-box annotations for every object that should be detected.
- Separate training, validation, and optional test splits.

Annotation quality rules

Good practice	Why it matters
Use consistent class names.	"car" and "vehicle" should not be mixed unless they are intentionally different classes.
Draw tight boxes around visible objects.	Loose or inconsistent boxes confuse localization learning.
Annotate all relevant objects.	Missing labels can teach the model that real objects are background.
Include negative images.	Images without target objects help control false positives.
Represent difficult conditions.	Night, blur, distance, occlusion, and unusual angles improve robustness.

Training workflow

18. Define the business objective and classes.
19. Collect representative images and remove unusable duplicates.
20. Annotate bounding boxes using a labeling tool.
21. Split data into training, validation, and test sets without leakage.
22. Choose a pretrained model and fine-tune it on the custom dataset.
23. Track training losses and validation metrics.
24. Inspect incorrect predictions, improve data, and retrain.
25. Export and benchmark the model on the target device.

Data augmentation examples

- Horizontal flipping when direction does not change the class meaning.
- Brightness, contrast, blur, or noise changes to simulate camera conditions.
- Scaling and cropping to improve object-size variation.
- Geometric transformations used carefully so boxes remain correct.

Important

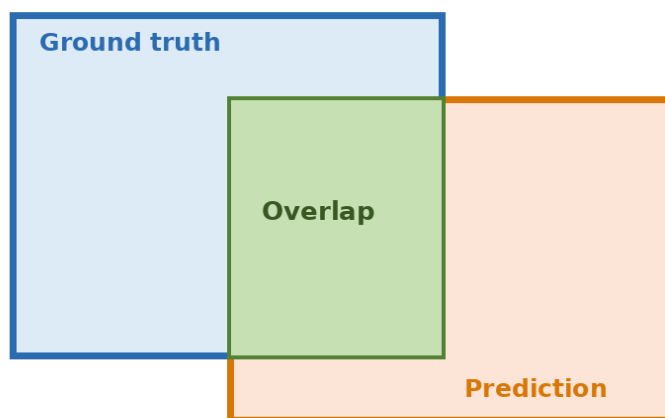
More data is not automatically better. Representative, correctly labeled, and diverse data usually matters more than collecting many near-duplicate images.

6. IoU and Non-Maximum Suppression

Intersection over Union (IoU)

IoU measures how much a predicted bounding box overlaps the ground-truth box. It is calculated as the intersection area divided by the union area. The value ranges from 0 to 1.

Intersection over Union (IoU)



IoU =

area of intersection

area of union

**Higher IoU = better
box alignment**

IoU value	Interpretation
0.00	No overlap.
0.30	Weak box alignment.
0.50	Moderate overlap; often used as a basic matching threshold.
0.80	Strong localization.
1.00	Perfect box overlap.

Non-Maximum Suppression (NMS)

A detector may predict several overlapping boxes for the same object. NMS keeps the strongest box and suppresses highly overlapping lower-confidence boxes.

26. Sort candidate boxes by confidence.
27. Keep the highest-confidence box.
28. Compare it with remaining boxes using IoU.
29. Remove boxes whose overlap is above the selected NMS IoU threshold.
30. Repeat until no candidate boxes remain.

Do not confuse the thresholds

Confidence threshold filters weak predictions. NMS IoU threshold controls duplicate suppression. Evaluation IoU threshold decides whether a prediction matches a ground-truth object.

7. Evaluation Metrics: Precision, Recall, AP, and mAP

Basic outcomes

Term	Meaning in object detection
True Positive (TP)	A predicted box correctly matches a real object with the correct class and sufficient IoU.
False Positive (FP)	The model predicts an object incorrectly, predicts the wrong class, or produces a poor box.
False Negative (FN)	A real object exists but the model misses it.

Precision and recall

Precision = $TP / (TP + FP)$
Recall = $TP / (TP + FN)$

Metric	What it answers	When it matters
Precision	Of all reported detections, how many were correct?	False alarms are costly, such as automated enforcement or defect rejection.
Recall	Of all real objects, how many were detected?	Missing an object is costly, such as safety equipment or medical screening.

AP and mAP

Average Precision (AP) summarizes the precision-recall relationship for a class. Mean Average Precision (mAP) averages AP across classes. COCO-style detection evaluation commonly reports AP averaged over multiple IoU thresholds from 0.50 to 0.95, alongside easier metrics such as AP at IoU 0.50.

Metric label	Meaning
AP50	Average precision when a prediction is considered matched at IoU 0.50.
AP75	A stricter localization requirement using IoU 0.75.
mAP50-95	AP averaged across classes and IoU thresholds from 0.50 to 0.95.

Practical warning

A single mAP number does not reveal every problem. Always inspect per-class AP, precision, recall, confusion patterns, small-object performance, and real prediction images.

8. Python Example with Ultralytics YOLO

The following example follows the current Ultralytics Python interface documented in July 2026. Model names can evolve, so check the official documentation when starting a new project.

Install

```
pip install ultralytics
```

Run prediction on an image

```
from ultralytics import YOLO

model = YOLO("yolo26n.pt")
results = model.predict(
    source="street.jpg",
    conf=0.25,
    imgsz=640,
    save=True,
)

for result in results:
    for box in result.bboxes:
        class_id = int(box.cls[0])
        label = model.names[class_id]
        score = float(box.conf[0])
        xyxy = box.xyxy[0].tolist()
        print(label, round(score, 3), xyxy)
```

What each part does

Code	Purpose
YOLO("yolo26n.pt")	Loads a pretrained lightweight detection checkpoint.
model.predict(...)	Runs inference on the specified source.
conf=0.25	Removes predictions below the confidence threshold.
imgsz=640	Sets the target inference image size.
save=True	Saves an annotated output image.
result.bboxes	Provides predicted bounding boxes and associated values.

Video or webcam sources

```
# Webcam: source=0
for result in model.predict(source=0, stream=True):
    annotated_frame = result.plot()

# Video: replace 0 with 'traffic.mp4'
```

Hinglish summary

Model load karo, source do, confidence threshold set karo, prediction result se boxes/classes nikalo, aur annotated output ko display ya save karo.

9. Real-World Use Cases, Challenges, and Best Practices

Common use cases

Domain	Example
Retail	Product counting, shelf monitoring, empty-slot detection, checkout automation.
Manufacturing	Defect detection, component verification, safety compliance.
Traffic	Vehicle detection, lane analytics, congestion measurement, parking occupancy.
Security	Intrusion detection, restricted-zone monitoring, unattended-object alerts.
Agriculture	Fruit counting, disease-region detection, animal monitoring.
Healthcare	Finding candidate regions in scans; requires strict validation and expert oversight.

Major challenges

- Small objects occupy very few pixels and may disappear after resizing.
- Occlusion hides part of an object behind another object.
- Domain shift occurs when production images differ from training data.
- Class imbalance gives common classes much more training influence.
- Annotation inconsistency creates noisy supervision.
- Real-time deployment adds latency, memory, power, and hardware constraints.
- Privacy, fairness, and misuse risks must be considered for people-related systems.

Best practices

- Define measurable acceptance criteria before selecting a model.
- Create a difficult validation set that resembles production.
- Examine failure cases instead of relying only on mAP.
- Benchmark end-to-end latency, including image loading and post-processing.
- Version datasets, annotations, code, configuration, and model weights.
- Monitor drift and false positives after deployment.
- Use human review for high-impact decisions.

10. Mini Project and Interview Revision

Mini project: Retail Shelf Product Detector

Goal: detect three product categories on a shop shelf and return the number of visible items for each category.

Phase	Tasks	Deliverable
1. Define	Choose 3 classes and clear labeling rules.	Class list and acceptance criteria.
2. Collect	Capture varied shelf images from different angles and lighting.	A representative image dataset.
3. Annotate	Draw boxes for every visible target product.	Verified annotations.
4. Train	Fine-tune a small pretrained detector.	Best model checkpoint.
5. Evaluate	Measure mAP, precision, recall, and per-class errors.	Evaluation report and failure examples.
6. Demo	Build an image-upload API or Streamlit interface.	Working detection application.

Suggested project folder structure

```
retail_detector/  
|-- data/  
|   |-- images/  
|   |   |-- train/  
|   |   |-- val/  
|   |-- labels/  
|   |   |-- train/  
|   |   |-- val/  
|   |-- dataset.yaml  
|-- notebooks/  
|-- src/  
|   |-- train.py  
|   |-- predict.py  
|   |-- evaluate.py  
|-- models/  
|-- outputs/  
-- requirements.txt
```

Interview-ready questions

Use these answers for quick revision. In an interview, add a small project example after the definition when the interviewer asks for more detail.

Question	Strong short answer
What is object detection?	It identifies object classes and locates every detected object using bounding boxes.
Detection vs classification?	Classification returns a label for the whole image; detection returns multiple labels and locations.
What is IoU?	The intersection area divided by the union area of two boxes.
Why is NMS used?	To remove duplicate overlapping predictions for the same object.
Precision vs recall?	Precision focuses on false positives; recall focuses on missed objects.
What is mAP?	The mean of average precision values across classes, often also averaged across IoU thresholds.
One-stage vs two-stage?	One-stage detectors predict directly and are typically faster; two-stage detectors first propose regions and are usually heavier.
How do you improve a detector?	Improve labels and data coverage, inspect errors, tune thresholds, select suitable image size/model, and retrain.

Final revision sheet

Remember this flow

Image -> preprocessing -> feature extraction -> box/class prediction -> confidence filtering -> NMS -> final detections -> evaluation using IoU, precision, recall, AP, and mAP.

References and Further Learning

The conceptual explanations in this guide use standard computer-vision terminology. The practical API example and selected metric details were checked against these official resources:

31. Ultralytics YOLO - Python Usage documentation. <https://docs.ultralytics.com/usage/python/>
32. Ultralytics YOLO - Predict Mode documentation. <https://docs.ultralytics.com/modes/predict/>
33. OpenCV - Deep Neural Networks tutorials.
https://docs.opencv.org/4.x/d2/d58/tutorial_table_of_content_dnn.html
34. COCO - Common Objects in Context dataset. <https://cocodataset.org/>
35. COCO Detection Evaluation overview - AP averaged across IoU 0.50 to 0.95.
<https://presentations.cocodataset.org/COCO17-Detect-Overview.pdf>

Next learning step

After understanding this guide, practice by running a pretrained detector on 10 images. Then review every false positive and missed object before attempting custom training.